

TECA2 – Atividade de Visão Computacional

Reconhecimento de Padrões em Cenas de Inspeção

Esta atividade fecha o último tópico em Visão Computacional. Na aula anterior tratamos da **busca** de padrões (*template matching*) e **classificação** por MLPs e aprendizado profundo (CNN). Agora montamos o **pipeline completo de reconhecimento** sobre uma cena binária: segmentação → descrição → classificação, e em seguida confrontamos os classificadores que vimos em classe. Você pode usar o Gonzalez em português apenas como referência; alguns temas estão melhor cobertos no Gonzalez novo (inglês) e nos demais livros citados.

As duas partes trabalham o **mesmo problema** e o **mesmo dataset**, dando continuidade entre elas: a 1ª Parte constrói o sistema clássico (segmentação, rejeição e classificador de distância mínima) e a 2ª Parte mantém o problema, mas troca o classificador por redes convolucionais, comparando custo, robustez e desempenho.

Período da atividade: de 31/maio/2026 a 27/junho/2026. **Dataset oficial:** o conjunto é gerado por um notebook fornecido (TECA2_Gerador_Dataset_Oficial.ipynb) e distribuído como .zip no Classroom. A aquisição é simulada em tons de cinza com *anti-aliasing* (*supersampling*), de modo que a variação de $\pm 10\%$ dos eixos seja contínua e não colapse na grade de pixels; a **binarização** passa a ser o primeiro passo do pipeline da 1ª Parte. Para as cenas, além da imagem em tons de cinza, são fornecidos uma versão binarizada (Otsu), um **mapa de instâncias** (id do objeto por pixel) e um mapa de contagem, que viabilizam a avaliação quantitativa.

1 Problema-base (Gonzalez) e cenário de aplicação

O problema-base (Gonzalez, Cap. 12; Problema 12.33; p. 993) é o do experimento que produz **imagens de blobs quase elípticos** (adquiridas em tons de cinza e binarizadas no pré-processamento). Os blobs ocorrem em **três tamanhos**, definidos pelos valores médios dos eixos principais das elipses:

Tabela 1: Classes e eixos principais médios.

Classe	Eixos principais médios (a, b)
A (grande)	(1,3; 0,7)
B (média)	(1,0; 0,5)
C (pequena)	(0,75; 0,25)

As dimensões dos eixos variam $\pm 10\%$ em torno desses valores médios. O sistema deve **rejeitar elipses incompletas ou sobrepostas** e, em seguida, **classificar as elipses simples remanescentes** em uma das três classes de tamanho. A solução deve ser apresentada em **diagrama de blocos**, com detalhes específicos da operação de cada bloco, e a classificação resolvida com um **classificador de distância mínima**, indicando claramente como obter as amostras de treino e como treiná-lo. A Figura 1 é adaptada do problema-base.



Figura 1: Exemplo de cena de inspeção com blobs quase elípticos, veja a original no Gonzalez.

Como cenário de aplicação sugerido, pense em **triagem/granulometria automatizada de partículas**

quase elípticas — por exemplo, células, esporos, microalgas, grãos ou granulado industrial sob aquisição binarizada. As três classes de tamanho correspondem às três populações de partículas; incompletas (cortadas no campo/borda) e sobrepostas (aglomeradas) devem ser descartadas antes de medir. Esse cenário será retomado nas duas partes.

Para dar uma restrição de tempo: suponha uma esteira de triagem a **30 cm/s** com partículas espaçadas **3 cm**, isto é, **~ 10 inspeções/segundo** ⇒ **inferência em menos de ~ 100 ms por partícula**. Reporte o tempo médio de inferência do seu sistema.

2 1ª Parte – Segmentação, Análise de Blobs e Classificador de Distância Mínima

2.1 Leitura

- Gonzalez and Woods 2017, Cap. 12 — Seção 12.2 (padrões e classes de padrões) e Seção 12.3 (classificação por correspondência de protótipos → **classificador de distância mínima** e funções de decisão; vide slides de nossa aula).
- Gonzalez and Woods 2017, Cap. 11 (Extração de Características) — descritores regionais, **momentos centrais e de Hu**, e os **eixos principais / elipse equivalente** obtidos da matriz de covariância das coordenadas da região (autovalores e autovetores dos momentos de 2ª ordem). Vimos isso na atividade sobre Representação e Descrição.
- Gonzalez and Woods 2017, Cap. 9 (Morfologia) — abertura/fechamento para limpeza de ruído e separação leve de toques.
- Peter Corke 2023 — capítulo de extração de características de imagem: *region features* / análise de blobs, momentos e a **elipse equivalente** derivada dos momentos de 2ª ordem (orientação, excentricidade).

- **Burger and Burge 2022** — capítulo sobre regiões em imagens binárias: rotulação de componentes conexas (pseudocódigo), momentos centrais normalizados, orientação e excentricidade a partir dos momentos.
- **Documentação OpenCV** — funções já implementadas e otimizadas:
 - `cv2.connectedComponentsWithStats`, `cv2.findContours`, `cv2.moments`, `cv2.HuMoments`, `cv2.fitEllipse`;
 - https://docs.opencv.org/4.x/d3/dc0/group_imgproc__shape.html;
 - https://docs.opencv.org/4.x/d3/dc1/tutorial_basic_geometric_drawing.html (geração das cenas sintéticas).
- **scikit-image** — `skimage.measure.regionprops` (`major_axis_length`, `minor_axis_length`, `eccentricity`, `solidity`, `euler_number`): <https://scikit-image.org/docs/stable/api/skimage.measure.html#skimage.measure.regionprops>.
- **Tutorial complementar** (segmentação/regiões, material muito bom, consultar se está ainda disponível): <http://csundergrad.science.uoit.ca/courses/cv-notes/notebooks/06-template-matching.html>.

2.2 Atividade prática

Leia e revise o material acima. Então:

1. **Dataset oficial.** Carregue o dataset oficial do .zip (gerado pelo notebook fornecido) e **documente os parâmetros**: SCALE, *supersampling*, $\pm 10\%$, tamanho dos recortes (32×32 em tons de cinza) e composição das cenas. Os recortes isolados (`dataset_blobs_isolados.npz`) vêm rotulados por classe; as cenas (`cenas_esteira.npz`) trazem objetos completos, cortados pela borda e sobrepostos/encostados, com `cenas_cinza`, `cenas_bin` (Otsu), `mapas_instancia` e `mapas_contagem`. Inspeção visualmente alguns exemplos. (Opcional: reexecute o gerador para auditar as escolhas de projeto.)
2. **Segmentação e rotulação.** Parta das cenas em tons de cinza e **binarize** (ex.: Otsu) — compare com a `cenas_bin` fornecida. Aplique morfologia mínima do Cap. 9 e rotule componentes conexas (`cv2.connectedComponentsWithStats`). Mostre a imagem rotulada (*label map* colorido).
3. **Módulo de rejeição** (o coração do exercício de Gonzalez). Defina e **justifique** critérios para descartar:
 - **incompletas**: contato com a borda da imagem (*border clearing*) e/ou área abaixo de um limiar;
 - **sobrepostas**: baixa **solidez** (`area / convex_area`), **resíduo alto do ajuste de elipse** (`cv2.fitEllipse` × região real), **buracos** / `euler_number`, **razão `area_real / area_da_elipse_ajustada`** fora de faixa, excentricidade anômala.
4. **Extração de características – eixos principais.** Implemente **do zero** o cálculo dos eixos principais a partir dos **momentos centrais de 2ª ordem** (Gonzalez/Burger): monte a matriz de covariância a partir de μ_{20} , μ_{02} , μ_{11} normalizados por μ_{00} e obtenha os autovalores $\lambda_1 \geq \lambda_2$. Converta para comprimento de eixo (por exemplo, eixo $\propto 4\sqrt{\lambda}$ para a elipse de mesma 2ª ordem — **verifique e documente a convenção** que você adotou). Calcule os momentos sobre a imagem **em tons de cinza** (ponderados por intensidade), o que recupera os eixos em precisão **sub-pixel** e preserva a variação de $\pm 10\%$; compare com a versão binarizada. **Compare** ainda sua medida com `cv2.fitEllipse` e com `regionprops` (`major/minor_axis_length`), discutindo discrepâncias e a conversão para o espaço de atributos (a, b) dos três protótipos.
5. **Classificador de distância mínima – implementação do zero.** Seguindo a Seção 12.3 do Gonzalez: vetor de atributos $x = (\text{eixo maior}, \text{eixo menor})$; protótipo de cada classe $m_j = \text{média das amostras de treino}$; função de decisão $d_j(x) = x^T m_j - \frac{1}{2} \|m_j\|^2$ (equivalente ao protótipo Euclidiano mais próximo). Explique **como obter as amostras de treino** (rotular blobs gerados com parâmetros conhecidos, ou rotular manualmente a partir das regiões segmentadas) e **como treinar** (estimar os m_j). Plote o **espaço de atributos 2D** com amostras, protótipos e **fronteiras de decisão** (mediatrizes). Valide contra `sklearn.NearestCentroid`.
6. **Diagrama de blocos do sistema.** Conforme exigido pelo problema, apresente o **pipeline completo em diagrama de blocos**, detalhando a operação de cada bloco: aquisição → pré-processamento/morfologia → segmentação/rotulação → **rejeição** → extração de momentos/eixos → **classificador de distância mínima** → decisão (classe A/B/C ou “rejeitado”).
7. **Avaliação.** Para montar a matriz de confusão quantitativa (3 classes + “rejeitado”), **alinhe** cada componente conexa ao *ground-truth* pelo `mapas_instancia`: componente que cobre **um único** id → objeto verdadeiro (classe/status conhecidos); componente que cobre **vários** ids → *merge* (deve ser rejeitado). Reporte a matriz de confusão e a acurácia global. Em vez de “erros de fronteira” (as três classes são linearmente separáveis com folga, mesmo com $\pm 10\%$), faça uma **análise de margem**: meça a distância entre nuvens/protótipos no espaço (a, b), mostre que não há sobreposição e explique por quê. A degradação só deve aparecer sob os testes de estresse (2ª Parte).

2.3 Entregáveis da 1ª Parte

- **1aParte.pptx** — apresentação com **um bloco/técnica por slide**: o diagrama de blocos, exemplos de segmentação e rejeição, o gráfico do espaço de atributos

tos com fronteiras, a matriz de confusão e a discussão completa/técnica dos resultados.

- **1aParte.ipynb** — notebook Colab caprichado, com *seeds* fixadas, células comentadas e resultados reproduzíveis.

Submeta os dois arquivos no Google Classroom. Se prepare para apresentar caso sorteado(a). Faça com capricho e seja didático(a).

3 2ª Parte – Classificadores de Redes Neurais sobre o Mesmo Problema

Mantenha **exatamente o mesmo dataset** da 1ª Parte. Aqui trocamos apenas o **classificador**, percorrendo a progressão do Cap. 12 do Gonzalez: MLPs e CNNs (temas da última aula).

3.1 Leitura

Para relembrar a disciplina de Inteligência Computacional, releia:

- **Gonzalez**, Seção 12.5 — redes neurais: perceptron, MLP e dedução da **retropropagação**.
- **Gonzalez**, Seção 12.6 — CNNs estilo LeNet; **Tabela 12.6** (passos para treinar uma CNN) e **Exemplo 12.17** (estrutura de referência).
- Use **PyTorch** (recomendo academicamente e por experiência de mercado).
- **PyTorch docs** — `nn.Conv2d`, `nn.Linear`, `autograd`; e as funções de ativação:
 - <https://pytorch.org/docs/stable/generated/torch.nn.GELU.html>;
 - <https://towardsai.net/p/1/is-gelu-the-relu-successor>;
 - <https://pytorch.org/docs/stable/generated/torch.nn.SiLU.html> (Swish/SiLU) ou Mish.

3.2 Atividade prática

1. **MLP sobre características.** MLP pequeno em PyTorch ($2 \rightarrow$ oculta $\rightarrow 3$) treinado sobre o vetor de atributos. *Forward/backward* via `autograd`, loop com *tracking* de *loss* e acurácia (treino $\times$ validação), *early stopping/checkpoint* do melhor modelo, e **split estratificado** treino/validação/teste (`train_test_split(stratify=...)`). Compare com distância mínima.
2. **CNN ponta-a-ponta.** CNN estilo LeNet (Tabela 12.6) que recebe o **recorte em tons de cinza do blob isolado** (tamanho fixo 32×32 , do `dataset_blobs_isolados.npz`) e classifica direto nas 3 classes — **aprendendo** as características em vez de usar os momentos desenhados à mão. Implemente *forward* (conv $\rightarrow$ pooling $\rightarrow$ conv $\rightarrow$ pooling $\rightarrow$ flatten $\rightarrow$ densas), *backward* via `autograd`, loop de treino

com *tracking* e *checkpoint*. Reporte curvas, matriz de confusão e acurácia de teste.

3. **Experimento comparativo – funções de ativação (na CNN).** Treine **três variantes** da mesma arquitetura, com **seeds idênticas**, trocando apenas a ativação:
 - **ReLU** (baseline);
 - **GELU**;
 - **Mish** ou **Swish/SiLU** (à sua escolha).

Para cada variante reporte: curvas de *loss* e acurácia (treino $\times$ validação), acurácia final de teste, matriz de confusão e **tempo médio por época**. Discuta se faz diferença prática neste problema.

4. **Confronto de abordagens.** Tabela comparando **distância mínima $\times$ MLP-sobre-features $\times$ CNN-ponta-a-ponta**: acurácia de teste, robustez, tempo de treino e **tempo de inferência por amostra**.
5. **Estresse / robustez.** Avalie todos os classificadores em entradas degradadas: ruído sal-e-pimenta, oclusão parcial, rotação extra, *jitter* de escala além de $\pm 10\%$ e suavização/anti-aliasing. Quem degrada de forma mais suave — os **momentos desenhados à mão** ou as **características aprendidas**?
6. **Interpretabilidade.** (a) Visualize os **feature maps da primeira camada convolucional** da CNN para 3 amostras de teste — o que cada filtro parece detectar (bordas, orientação, tamanho)? (b) Plote as **regiões de decisão** dos classificadores clássicos no espaço de atributos 2D. Confronte: o que a CNN aprende $\times$ o que os momentos codificam.
7. **Discussão – analogia com a linha de produção.** Custo $\times$ simplicidade: dado que as três classes são bem separáveis e os momentos são baratos de calcular, a CNN se justifica? Em que situações o aprendizado venceria (oclusão, variação maior, ruído, classes não-separáveis)? Reporte o **tempo de inferência por partícula** frente ao orçamento de ~ 100 ms da esteira.

3.3 Entregáveis da 2ª Parte

- **2aParte.pptx** — apresentação explicitando e ilustrando os passos da Tabela 12.6, com os gráficos dos experimentos comparativos e a tabela de confronto entre métodos.
- **2aParte.ipynb** — notebook Colab em PyTorch, com *seeds* fixadas para reprodutibilidade.

Submeta os dois arquivos no Google Classroom.

4 Critérios de avaliação

Tabela 2: Critérios de avaliação.

Critério	Peso
Capricho e clareza das apresentações (.pptx)	25%
Reprodutibilidade dos notebooks (<i>seeds</i> , dependências documentadas)	15%
Profundidade da discussão dos resultados	30%
Comparação quantitativa entre métodos/configurações	20%
Implementação manual da extração de momentos + classificador de distância mínima (1ª Parte)	10%

5 Observações finais

- **Prazo:** 31/maio/2026 a 27/junho/2026. Por padronização, use obrigatoriamente o **dataset oficial** do .zip; dados extras só como experimentos complementares por sua conta e de seu grupo, claramente identificados e com motivação bem explicada.
- Os dois notebooks devem rodar de ponta-a-ponta no Colab sem intervenção manual (incluindo `!pip install` na primeira célula).
- Documente as decisões de projeto: parâmetros do dataset (escala, *supersampling*, $\pm 10\%$), escolha dos limiares de rejeição, hiperparâmetros, e o que tentou fazer e descartou.
- O **diagrama de blocos é obrigatório** na 1ª Parte (exigência explícita do problema do Gonzalez).
- Para a matriz de confusão das cenas, use o `mapas_instancia` para alinhar componentes ao *ground-truth* (ver 1ª Parte, item de avaliação).
- Reporte o **tempo médio de inferência por partícula** e compare com o orçamento de ~ 100 ms da esteira.
- **Separabilidade.** Com as médias do Gonzalez e $\pm 10\%$, as três classes **não se sobrepõem** no espaço (a, b) — todo classificador tende a $\approx 100\%$ no caso limpo. O interesse da comparação está, portanto, no **custo** (a CNN se justifica se a distância mínima já resolve?) e na **robustez sob estresse** (ruído, oclusão, rotação/escala extras): é aí que momentos desenhados à mão e características aprendidas se diferenciam.